PostgreSQL-Centered Research Network

Architecture and implementation outline | 25 October 2026

Purpose

Use PostgreSQL as the coordination backbone for nanopublications, Roam research graphs, and Obsidian vaults. Preserve source material, track contributions, and support shared discovery without requiring every team to use the same workspace. This is a proposed architecture, not a turnkey implementation.

1. Ingest source structure, not just text

Roam Research: ingest structured JSON exports. Preserve pages, block UIDs, hierarchy, ordering, page links, block references, and available metadata. Do not use Markdown as the sole archive of a structured graph. Export fields vary; test against actual source files. [1]

Obsidian: ingest selected Markdown files, YAML properties, and referenced attachments. Preserve paths, properties, tags, links, embeds, headings, and explicit block identifiers. Support both Markdown links and Obsidian-style links. [2, 3]

Nanopublications: use an RDF-aware importer. Preserve identifiers, named graphs, assertions, provenance, publication information, and original serialization. [4, 5]

Assign central identifiers while retaining source identifiers. Namespace Roam block UIDs by source graph. For Obsidian, maintain persistent note identifiers rather than relying solely on paths. Record import time, parser version, fingerprints, and revisions. Repeated imports must not create duplicates. Start with one-way ingestion; write-back requires separate ownership and conflict rules.

2. Separate source records from interpretations

Use relational tables for publications, notes, passages, concepts, people, projects, annotations, and relationships. Use jsonb for variable metadata. Preserve exact originals in private object storage or byte-preserving database fields: jsonb does not retain whitespace, key order, or duplicate keys. [6]

Give each relationship an identity: source -> relationship -> target, with creator, project, time, evidence, and status. Distinguish imported links, human interpretations, machine suggestions, and inferred relationships. A backlink is a reference, not proof of agreement or support. Web Annotation and PROV provide models for interpretations and their origins. [7, 8]

For RDF exchange, preserve named graphs, literal datatypes, language tags, and blank-node identity. A generic links table is not a lossless RDF representation. Scope blank nodes to their source dataset. [5]

3. Apply shared knowledge frameworks

- ISO 25964: guide subject vocabulary design and mappings across teams. [9]

- SKOS: represent themes, broader/narrower relationships, related concepts, and vocabulary mappings. [10]

- Web Annotation: attach comments, tags, and interpretations to resources or passages. [7]

- PROV / PROV-O: track contributors, sources, and activities producing each contribution. [8]

- ISO/IEC 13250 Topic Maps: inform project-specific maps of subjects, resources, and context; this is an alternative modeling framework, not an RDF requirement. [11]

- RDF / OWL: support graph exchange and formal definitions where needed. Storage in PostgreSQL does not itself provide SPARQL execution or OWL reasoning. [5, 8]

Preserve original labels. Add explicit mappings instead of forcing one vocabulary. Document mapping limits; similar column names do not establish standards conformance.

4. Configure PostgreSQL for discovery and access control

- Full-text search: tsvector, tsquery, and GIN indexes for notes, passages, publications, and annotations. Configure language dictionaries. [12]

- pg_trgm: approximate title, name, and label matching; suggest duplicates without merging automatically. [13]

- pgvector, installed as vector: search externally generated embeddings. Record model and version; add HNSW indexes when justified. Test filtered-search recall and tune iterative scans where supported. [14]

- WITH RECURSIVE: traverse note hierarchies, citation chains, and theme relationships. [15]

- Apache AGE, optional: property-graph queries where justified; it is not an RDF/OWL reasoner. Check deployment compatibility. Do not assume availability on hosted Supabase. [16]

- Roles, grants, and Row-Level Security: separate personal, project, and public contributions. Privileged roles can bypass RLS; test real application roles. [17, 18]

5. Expose a shared research API

Build a versioned, authenticated API over permission-filtered queries. Use stable identifiers, pagination, bounded graph depth, revision checks, and documented response formats.

- Search: keyword, fuzzy, and semantic retrieval, filtered by project, source, contributor, date, and evidence status. Return matching passages, source links, and the type of match.

- Views: bounded graph neighborhoods, timelines, citation trails, and theme summaries. Keep layout and display logic outside PostgreSQL.

- Annotation: create comments, tags, evidence relationships, and competing interpretations. Require target identifiers, attribution, and optimistic concurrency checks.

- Provenance: expose origins, revisions, import runs, and whether a relationship was imported, proposed, accepted, or inferred.

- Synchronization and export: provide cursor-based change feeds, repeatable imports, and portable JSON/RDF exports. Document unsupported or lossy mappings.

The API gives a browser, an Obsidian integration, a Roam importer, and automated assistants the same permissions, identifiers, and history. Enforce access rules before returning search results, counts, snippets, or graph edges. Apply equivalent controls to caches, exports, and attachment downloads.

6. Add Maven-inspired discovery; keep integration optional

Maven's interest-based discovery and serendipity controls suggest a useful exploration mode: surface unexpected cross-topic connections alongside focused retrieval. A discovery endpoint can return proposed bridges, supporting material, and the signals behind each suggestion. [19]

Keep suggestions separate from accepted relationships. Embedding similarity is neither evidence nor Stanley-style novelty search by itself. Exploration policy is application logic.

A Maven API connector is conditional on documented access, authentication, permitted uses, and portability. No official public API or open-source release for Stanley's Maven social platform was verified for this proposal. Do not assume either exists, and do not confuse this product with Apache Maven. The local discovery service must work without it.

7. Favor open components and portable interfaces

PostgreSQL, pgvector, and Apache AGE provide an open-source foundation. Publish custom importers, schema mappings, and API contracts under explicit licenses where feasible. Supabase also offers a self-hosting route, but operational responsibilities differ from its hosted service. [14, 16, 24]

Open-source software, open standards, open APIs, and permission to redistribute research content are separate decisions. Preserve source ownership and access restrictions through every export.

Start with PostgreSQL, pg_trgm, pgvector, import adapters, revisions, and tested permissions. Add graph tooling or reasoning only for demonstrated requirements. Workspaces support research; PostgreSQL coordinates records; standards preserve meaning; APIs enable reuse; discovery services propose new paths.

Appendix A. Supabase pseudo-deployment guide

Deployment sequence, not a production-ready runbook. Verify extension versions, quotas, regional requirements, and backup options in the target project.

A1. Establish environments and boundaries

Create separate development, staging, and production projects. Use the Supabase CLI and version-controlled SQL migrations. Keep credentials out of the repository. Use Supabase Auth for users and an application membership table for projects. [18, 20]

Use a private schema for ingestion staging, raw RDF statements, and internal job records. Expose only deliberate tables, views, and RPC functions through the Data API. Private-schema placement alone does not replace grants and authorization checks.

A2. Apply schema and extension migrations

Enable vector and pg_trgm if available in the project's extension catalog. Full-text search and recursive SQL are core PostgreSQL features. Do not include AGE in the baseline deployment; verify support separately. [12-16, 21]

Suggested tables: workspaces, projects, memberships, source_items, revisions, passages, concepts, concept_mappings, relationships, annotations, import_runs, change_events, embedding_jobs, and embeddings. Add rdf_terms and rdf_quads if faithful RDF interchange is required.

Enforce unique source identities, foreign keys, and revision constraints. Index project membership, source identity, relationship endpoints, timestamps, searchable text, and selected JSONB paths. Set embedding dimensions to the chosen model before creating vector indexes.

A3. Protect rows, files, and functions

Enable RLS and least-privilege grants in migrations. Define separate read/write policies for memberships, source items, annotations, and dependent passages or embeddings. Test allow and deny cases for each operation. [18]

Use the publishable key and user JWT in clients. Backend secret/service-role credentials bypass RLS: never put them in clients. Elevated workers must independently validate project scope. Prefer invoker-security RPCs; audit any security-definer function and fix its search_path. [18, 22]

Create private Storage buckets for source archives and attachments. Authorize uploads and downloads, and issue short-lived signed URLs only after access checks. Treat attachment permissions as part of the research record. [23]

A4. Run import and enrichment workers

Run parsers in a backend worker with file-size limits, safe YAML parsing, archive limits, and retryable jobs. Use short serverless handlers for authentication and job submission, not unbounded parsing workloads.

Stage each import, validate it, then commit revisions and change events transactionally. Generate embeddings only after an approved data-handling decision; external embedding providers may receive confidential text. Keep model/version metadata.

A5. Publish the API

Use Supabase's generated Data API for simple authorized CRUD. Add versioned application endpoints or database RPCs for hybrid search, bounded graphs, annotation revisions, and exports. [25]

Example contract: GET /v1/search; GET /v1/items/{id}; GET /v1/graphs/neighborhood; GET /v1/projects/{id}/timeline; POST /v1/annotations; PATCH /v1/annotations/{id}; GET /v1/changes; POST /v1/imports; POST /v1/discovery/suggestions; GET /v1/exports/{id}.

The routes are proposed application endpoints, not built-in Supabase routes. Use authenticated, paginated responses; enforce result limits. Return conflict responses on stale revisions. Realtime notifications may refresh interfaces, but use a durable change log for resumable synchronization.

A6. Verify operations before release

Test cross-project isolation, renamed files, ambiguous references, deleted sources, partial imports, duplicate uploads, stale edits, and vector-search filtering. Confirm backups and restore procedures for both the database and Storage objects: database backups do not include Storage object contents. [20]

Pin dependencies and document licenses. Monitor import failures, unresolved links, search latency, denied access, and index size. Keep Maven integration disabled until access and terms are verified.

Appendix B. Draft README.md

The following is proposed repository documentation. Names and commands are illustrative; the application and workers still need implementation.

```
# Research Network

A PostgreSQL/Supabase backbone for nanopublications,
Roam graphs, and Obsidian vaults.

## Status
Architecture prototype. One-way imports first.
No assumed Maven API access. No built-in OWL reasoner.

## Components
- Supabase: PostgreSQL, Auth, private Storage, Data API
- Import worker: Roam JSON, Obsidian Markdown, RDF
- API: search, annotations, graphs, history, exports
- Discovery worker: optional embeddings and suggestions

## Requirements
- Supabase CLI and container runtime for local development
- Supported PostgreSQL with vector and pg_trgm
- Worker runtime and pinned dependency lockfile
- RDF parser, safe YAML parser, Markdown parser
- Optional approved embedding provider/model

## Proposed layout
supabase/migrations/     schema, grants, policies, indexes
supabase/tests/          authorization and integrity tests
importers/roam/          JSON parsing and block resolution
importers/obsidian/      vault parsing and link resolution
importers/rdf/           RDF import and graph preservation
api/                    versioned application endpoints
workers/                imports, embeddings, exports
schemas/                mappings and API contracts
fixtures/               synthetic, non-confidential samples

## Local setup
supabase init            # once, if not initialized
supabase start
supabase db reset        # local only; destructive
supabase test db
# Start implemented API and worker using project scripts.
# Configure secrets locally; never commit them.

## Configuration
SUPABASE_URL
SUPABASE_PUBLISHABLE_KEY # client-safe only with policies
SUPABASE_SECRET_KEY      # backend only; bypasses RLS
EMBEDDING_MODEL          # optional; fixes vector dimensions
EMBEDDING_PROVIDER_KEY   # optional; backend only
MAVEN_CONNECTOR_ENABLED=false

## Import contract
1. Choose authorized workspace and project.
2. Upload source archive to private Storage.
3. Submit import job; receive run ID.
4. Parse, validate, resolve links, commit revisions.
5. Report counts, warnings, unresolved references.
6. Re-import safely; do not infer deletion from partial input.

## API contract
All routes are proposed and require implementation.
Authenticate requests; enforce membership and RLS.
Paginate results; bound graph depth and export size.
Use expected_revision on edits; reject stale updates.
Expose source links, provenance, and suggestion status.

## Standards
ISO 25964 / SKOS: themes and vocabulary mappings
Web Annotation: targeted interpretations
PROV-O: attribution and activity history
Topic Maps: contextual-view design guidance
RDF/OWL: interchange and formal definitions
Storage alone does not establish standards conformance.

## Security and tests
No backend secret in browser or desktop plugins.
Test row, file, search, export, and graph isolation.
Do not send private notes to an embedding provider
```

```
without explicit authorization.
Back up database AND Storage objects; test restores.

## Portability and licensing
Export original files, structured JSON, and mapped RDF.
Document lossy conversions and unresolved links.
Choose a license for custom code before distribution.
Respect upstream licenses and source-content rights.
```

Appendix C. Importer/parser pseudocode

The pseudocode specifies behavior, not a particular language or library. Both importers create normalized records; they do not silently promote source links into SKOS or scientific assertions.

C1. Shared import pipeline

```
function import_snapshot(actor, workspace, project, input):
    authorize(actor, workspace, project, "import")
    validate_size_type_and_archive_paths(input)
    run = record_import_run(actor, parser_version, input_hash)
    save_original_to_private_storage(input)
    parsed = adapter_for(input).parse_into_staging(input)
    validate_source_ids_and_required_fields(parsed)
    resolve_links_in_second_pass(parsed)
    transaction:
        for item in parsed.items:
            key = (workspace.id, item.source_identity)
            existing = find_by_source_key(key)
            if existing.content_hash == item.content_hash:
                continue
            write_new_item_or_revision(existing, item)
            record_import_activity_and_change_event(run, item)
        upsert_resolved_edges_and_record_unresolved_edges()
        if parsed.is_complete_snapshot and deletion_is_approved:
            tombstone_missing_items_without_erasing_history()
        mark_run_committed(run)
    enqueue_authorized_search_and_embedding_updates()
    return counts_warnings_and_unresolved_references(run)

# Failure: keep diagnostics, leave live data consistent.
# Retry: reuse idempotency keys; do not duplicate revisions.
# Completion means all expected input was read successfully.
```

C2. Roam JSON importer

Preserve the nested block structure and resolve references only after all blocks are known. Treat export timestamps as source metadata, not verified authorship. Roam fields and reference syntax require fixture-based testing. [1]

```
function parse_roam(json_export, workspace):
    pages = parse_json_with_limits(json_export)
    index = new_source_index()
    for page in pages:
        page_key = namespace(workspace, page.uid)
        stage_page(page_key, page.title, raw=page)
        walk_blocks(page.children, parent=page_key)

    function walk_blocks(blocks, parent):
        for position, block in enumerate(blocks):
            key = namespace(workspace, require(block.uid))
            stage_block(key, text=block.string,
                        parent=parent, order=position,
                        source_metadata=available_metadata(block))
            index.add(key)
            refs = parse_roam_references(block.string)
            stage_refs(key, refs, kind_and_original_syntax=True)
            walk_blocks(block.children or [], key)

    for ref in staged_refs:
        resolve_block_uid_or_page_title(ref, workspace, index)
        # Keep ambiguity or missing targets explicit.
        # Preserve embeds separately; do not duplicate content.
    return staged_snapshot
```

C3. Obsidian vault importer

Preserve Markdown, properties, and attachments. Do not execute plugins, embedded scripts, or downloaded content. Parse Markdown structure rather than relying on a single link-matching expression. [2, 3]

```
function parse_obsidian(vault, workspace, selection):
    files = safely_list_selected_files(vault, selection)
    registry = load_workspace_identity_registry(workspace)
    for file in markdown_files(files):
        raw = read_utf8_with_limits(file)
        properties, body = safe_parse_frontmatter(raw)
        note_id = persistent_property_or_registry_id(file, properties)
        # Match renames only when reliable; flag ambiguity.
```

```
    stage_note(note_id, path=file.path, raw=raw,
               properties=properties)
    ast = parse_markdown_with_obsidian_link_support(body)
    for section_or_block in ast:
        stage_passage(note_id, explicit_block_id_if_present,
                      heading_path, source_offsets, text)
        # Generated passage IDs may change after edits;
        # keep old revision targets rather than guessing.
    stage_tags_aliases_links_and_embeds(ast, properties)

build_path_title_alias_and_block_indexes()
for ref in staged_refs:
    resolve_relative_path_or_wikilink(ref)
    resolve_heading_or_explicit_block_target(ref)
    record_missing_or_ambiguous_target(ref)
for attachment in authorized_referenced_attachments:
    validate_path_type_and_size(attachment)
    preserve_private_blob_and_content_hash(attachment)
return staged_snapshot
```

References

Numbered citations apply throughout the main document and appendices. Product documentation is maintained continuously; verify against the deployed versions. Some implementation links are provided as further configuration references.

[1] Roam JSON export format (David Bieber; export-field reference).
https://davidbieber.com/snippets/2020-04-25-roam-json-export/

[2] Obsidian Help: Properties.
https://help.obsidian.md/properties

[3] Obsidian Help: Internal links.
https://help.obsidian.md/links

[4] Nanopublication Guidelines, working draft.
https://nanopub.net/guidelines/working_draft/

[5] W3C: RDF 1.1 Concepts and Abstract Syntax.
https://www.w3.org/TR/rdf11-concepts/

[6] PostgreSQL: JSON Types.
https://www.postgresql.org/docs/current/datatype-json.html

[7] W3C: Web Annotation Data Model.
https://www.w3.org/TR/annotation-model/

[8] W3C: PROV-O, The PROV Ontology.
https://www.w3.org/TR/prov-o/

[9] NISO: ISO 25964, thesauri and interoperability.
https://www.niso.org/schemas/iso25964

[10] W3C: SKOS Reference.
https://www.w3.org/TR/skos-reference/

[11] ISO: ISO/IEC 13250:2003, Topic Maps.
https://www.iso.org/standard/38068.html

[12] PostgreSQL: Full Text Search; see also index guidance.
https://www.postgresql.org/docs/current/textsearch.html

[13] PostgreSQL: pg_trgm.
https://www.postgresql.org/docs/current/pgtrgm.html

[14] pgvector: official repository and configuration documentation.
https://github.com/pgvector/pgvector

[15] PostgreSQL: WITH Queries, including recursive queries.
https://www.postgresql.org/docs/current/queries-with.html

[16] Apache AGE: project overview.
https://age.apache.org/

[17] PostgreSQL: Row Security Policies.
https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[18] Supabase: Row Level Security.
https://supabase.com/docs/guides/database/postgres/row-level-security

[19] TechCrunch (2024): Maven serendipity network.
https://techcrunch.com/2024/05/26/how-mavens-ai-run-serendipity-network-can-make-social-media-interesting-again/

[20] Supabase: Database overview and backup scope.
https://supabase.com/docs/guides/database/overview

[21] Supabase: Postgres extensions.
https://supabase.com/docs/guides/database/extensions

[22] Supabase: Securing your data and client/backend keys.
https://supabase.com/docs/guides/database/secure-data

[23] Supabase: Storage access control.
https://supabase.com/docs/guides/storage/security/access-control

[24] Supabase: Self-hosting.
https://supabase.com/docs/guides/self-hosting

[25] Supabase: Data REST API.
https://supabase.com/docs/guides/api